SMIT FINAL HACKATHON

MaintainIQ

AI-Powered QR Maintenance & Asset History Platform

Scan. Report. Diagnose. Maintain.

One product theme. Multiple difficulty tracks. Eight hours to demonstrate product thinking, engineering ability, AI integration, debugging skill, and ownership of code.

Audience	Track
Batch 16 & 17 - Final Hackathon Students	Advanced Full-Stack + GenAI
Batches 18-20 with Supabase/Firebase	Backend-as-a-Service
HTML, CSS & JavaScript Students	Frontend Prototype

Duration: 8 Hours

Deployment: Any platform allowed | Bonus for AWS + Docker + GitHub Actions

1. Hackathon Overview

MaintainIQ is a professional maintenance-management platform that gives every physical asset a digital identity, a QR-accessible public page, an issue-reporting workflow, and a permanent service history. The product is designed for schools, universities, hospitals, offices, factories, housing societies, labs, hotels, restaurants, warehouses, and facility-management companies.

The hackathon is not about generating QR codes. The QR code is only the entry point. The actual product value lies in issue triage, assignment, maintenance workflow, evidence, history, accountability, and preventive recommendations.

Product Problem

In many organizations, maintenance requests are scattered across registers, phone calls, WhatsApp messages, and spreadsheets. Teams struggle to answer basic questions:

- Which assets are repeatedly failing?
- Who reported the issue and when?
- Who is responsible for fixing it?
- What action was performed previously?
- Which parts were replaced?
- When is the next service due?
- Which assets are unsafe or out of service?

Product Solution

MaintainIQ centralizes the complete lifecycle of an asset:

Stage	Product Action
Asset Registration	Create a digital record and unique asset code.
QR Access	Open a safe public asset page by scanning or opening its link.
Issue Reporting	Submit a problem with description, priority, and optional evidence.
AI Triage	Generate structured issue details, possible causes, and initial checks.
Assignment	Assign the issue to an appropriate technician.
Maintenance	Record inspection, actions, parts, cost, and evidence.
Resolution	Resolve or reopen issues through a controlled status workflow.
Asset History	Preserve a permanent timeline of meaningful activity.
Preventive Action	Recommend the next service or identify recurring failures.

2. Expected Product Demonstration

The final product should be capable of demonstrating the following scenario:

1. An administrator registers “Classroom Projector 01” and the system generates a unique code and QR-accessible link.
2. A user opens the public asset page and reports: “The projector display is flickering and sometimes does not detect HDMI.”
3. The AI Issue Triage feature suggests a professional title, category, priority, possible causes, and initial diagnostic checks.
4. The user reviews or edits the AI suggestions before submitting the issue.

5. The issue appears on the internal dashboard and is assigned to a technician.
6. The technician starts inspection, records that the HDMI cable is damaged, and adds maintenance notes.
7. The technician records the replacement part and marks the issue as resolved.
8. The asset returns to Operational status and its permanent history is updated.
9. The system provides a professional AI-generated maintenance summary or preventive recommendation.

3. Product Users and Roles

Role	Primary Responsibilities
Administrator	Register assets, edit asset data, view all issues, assign technicians, schedule maintenance, review analytics, and control organization-level settings.
Technician	View assigned work, start inspection, add maintenance notes, record parts and cost, upload evidence, and resolve assigned issues.
Reporter / Public User	Open an asset page, view safe asset information, report an issue, and optionally check the reported issue status.
Supervisor (Optional)	Review completed maintenance, approve resolution, reopen issues, and monitor team performance.

4. Core Product Modules

4.1 Authentication and Role Access

- Register and login where required by the assigned track.
- Protect internal pages and actions.
- Enforce authorization on the server for advanced students.
- Public asset pages must expose only safe information.

4.2 Asset Management

- Create, view, edit, search, filter, and inspect assets.
- Every asset must have a unique code.
- Record category, location, condition, status, dates, and assigned technician.
- Display last service and next service dates.

4.3 QR Code Generation

- Generate a working QR code automatically after each asset is created.
- Encode only the asset's safe public URL inside the QR code; never encode private notes, serial data, internal costs, or user information directly.
- Show the QR code on the asset details screen and provide QR preview, download, copy-link, and Open Public Asset Page actions.
- Provide a print-ready asset label containing the organization name, asset name, asset code, location, QR code, and a short scan instruction.
- Physical printing is not required during the hackathon. The evaluator may scan the QR directly from a laptop screen using a mobile phone.
- Each QR code must map to exactly one asset. Editing an asset name or location must not break the QR mapping.
- Invalid asset identifiers must show a proper not-found state. Retired assets should remain readable but must clearly display Retired status.
- Optional innovation: allow administrators to select multiple assets and generate a bulk QR label sheet.

4.4 Public Asset Access

- Scanning the QR code or opening the public URL must display a safe, mobile-friendly asset page without requiring internal login.
- Show asset name, asset code, category, location, current condition, current status, last service date, next service date, safe recent activity, and a Report Issue action.
- Do not expose private technician notes, administrative controls, confidential serial information, sensitive costs, internal attachments, or private user details.
- The public page must remain linked to the same asset even when editable display fields such as name or location are changed.

4.5 Issue Reporting

- Report an issue against a specific asset.
- Capture title, description, priority, category, reporter information, and optional evidence.
- Generate a unique issue number.
- Update the asset status when an issue is submitted.

4.6 AI Issue Triage

- Convert a natural-language complaint into structured information.
- Suggest title, category, priority, possible causes, and safe initial checks.
- Allow users to edit or reject AI suggestions before saving.
- Handle AI timeout, invalid output, or unavailable service gracefully.

4.7 Assignment and Maintenance Workflow

- Assign issues to technicians.
- Track status transitions from reported to resolved.
- Record inspection findings, work performed, parts, cost, time, evidence, and final condition.
- Prevent invalid status transitions.

4.8 Asset History

- Automatically record significant events.
- Show date, actor, action, and related issue.
- History should not be casually editable or deletable.

4.9 Search, Filters, and Dashboard

- Search assets and issues.
- Filter by status, category, location, technician, and priority where applicable.
- Show useful maintenance summary cards, not merely decorative charts.

5. Required Business Rules

5.1 Asset Statuses

Recommended asset statuses: Operational, Issue Reported, Under Inspection, Under Maintenance, Out of Service, and Retired.

Event	Expected Asset Status
New issue submitted	Issue Reported
Technician begins inspection	Under Inspection

Repair work begins	Under Maintenance
Maintenance successfully completed	Operational
Critical safety issue identified	Out of Service
Asset permanently removed	Retired

5.2 Issue Statuses

Recommended issue statuses: Reported, Assigned, Inspection Started, Maintenance In Progress, Waiting for Parts, Resolved, Closed, and Reopened.

- A technician may update only an issue assigned to them, unless the chosen track does not include roles.
- A resolved issue may be reopened.
- A closed issue may not be edited until reopened.
- A critical issue must be visually distinguishable.
- An issue should not be resolved without a maintenance note.
- Maintenance cost cannot be negative.
- Next service date cannot be before the maintenance completion date.
- Duplicate asset codes must be rejected.
- Important actions must create history records.

6. GenAI Integration

GenAI must be used as a focused product capability, not as a generic chatbot. The preferred mandatory capability for the Advanced Track is AI Issue Triage.

6.1 Mandatory Advanced AI Capability: AI Issue Triage

The user provides a natural-language complaint. The system sends relevant asset context and receives structured output.

Input Context	Expected AI Output
Asset type, model, condition, location, recent history, and user complaint	Professional issue title
Complaint description	Suggested category and priority
Asset context	Possible causes
Safety context	Safe initial diagnostic checks
History where available	Recurring-pattern warning

Example user input:

“The AC is leaking water, making unusual noise, and cooling is weak.”

Example structured output:

Title: Water leakage and reduced cooling
Category: Leakage / Performance
Priority: High
Possible Causes: Blocked drain pipe, dirty filter, frozen coil
Initial Checks: Turn off the unit if water is near electrical wiring; inspect drainage; check filter condition.

6.2 Optional AI Enhancements

- AI Maintenance Summary: Convert rough technician notes into a professional service report.
- AI Asset Health Analysis: Detect repeated issue patterns in an asset history.
- AI Preventive Recommendation: Suggest next inspection or replacement consideration.
- AI Similar-Issue Finder: Surface related previous issues and their resolutions.
- AI Report Generator: Produce a structured maintenance completion report.
- AI Multilingual Complaint Assistant: Convert Roman Urdu or Urdu complaints into structured English records.

6.3 AI Quality and Safety Requirements

- AI output is advisory; the final decision belongs to a human user.
- Users must be able to review and edit generated values before saving.
- API keys must never be exposed in frontend code.
- Use structured JSON output or validate the response before storing it.
- Provide loading, timeout, retry, fallback, and error states.
- Do not provide unsafe instructions for electrical, mechanical, fire, medical, or industrial hazards.
- Clearly recommend qualified technicians for critical or safety-related issues.
- Store whether a field was AI-suggested and whether the user edited it, where practical.

7. Student Tracks

All students work on the same product theme, but each group receives a scope appropriate to its current skills. Students are evaluated only against the requirements of their assigned track.

Track A - Batch 16 & 17: Advanced Full-Stack + GenAI

Recommended technologies: Node.js with NestJS or Express.js; React or Next.js; JavaScript or TypeScript; MongoDB, PostgreSQL, or another suitable database.

Mandatory Scope

- Authentication and at least Admin and Technician roles.
- Backend-enforced authorization.
- Asset registration with unique asset code.
- Asset list, details, search, and filters.
- Automatic QR generation linked to a secure public asset route, with QR preview, downloadable QR/label, and copyable public link.
- Public issue-reporting page.
- Issue assignment and controlled status workflow.
- Maintenance record with notes, parts, cost, dates, and evidence.
- Cloudinary or equivalent cloud media storage for images/video evidence.
- Permanent asset history timeline.
- AI Issue Triage with structured output and human review.
- Responsive frontend and working deployment.
- Clear README, API documentation, and demo credentials.

Advanced Bonus Capabilities

Bonus Capability	Examples
AWS Deployment	EC2, ECS, Elastic Beanstalk, Lambda, RDS, S3, or appropriate AWS service.
Docker	Containerized client/server and repeatable environment.
GitHub Actions	Automated test, build, image push, or deployment pipeline.
Redis	Caching, job coordination, rate-limit storage, or another justified use.
Email	OTP, issue-assignment alert, resolution notice, or scheduled-maintenance reminder.
Rate Limiting	Public reporting endpoint, auth endpoints, AI endpoint, or upload endpoint.
Realtime	Socket.IO or equivalent for issue/status updates.
Additional AI	Maintenance summaries, history analysis, similarity search, or multilingual assistance.

Bonus architecture receives marks only when it is functional and justified. An incomplete core product with Kafka or Redis is weaker than a complete product with clean engineering.

Track B - Supabase/Firebase Students

Students who know HTML/CSS/JavaScript or a frontend framework together with Supabase or Firebase should build a persistent, multi-user or single-owner product using the services available to them.

Mandatory Scope

- Authentication using Supabase Auth or Firebase Authentication.

- Create and manage assets.
- Unique asset code, generated QR code, and public/open asset page linked to the correct asset.
- Report issues against an asset.
- Persist assets, issues, and maintenance records in the available database.
- Update issue status and add technician/work notes.
- Upload evidence using available storage or a justified alternative.
- Show an asset history or activity timeline.
- Search or filter assets/issues.
- Responsive deployed application.

AI Requirement

GenAI is optional but encouraged. API keys must not be exposed in client-side code. Students may use a Supabase Edge Function, Firebase Cloud Function, trainer-provided endpoint, or a safe mocked AI response. A rule-based issue classifier is acceptable only when the trainer has not covered secure AI API integration.

Recommended Bonus Features

- Realtime database updates
- Email or notification workflow
- Scheduled maintenance reminders
- AI maintenance summary
- Public issue status tracking
- Simple analytics
- Downloadable or bulk QR label generation

Track C - HTML, CSS & JavaScript Students

Students without database knowledge will create a frontend prototype using localStorage. They should demonstrate user experience, JavaScript logic, DOM manipulation, arrays/objects, filtering, validation, and persistence.

Mandatory Scope

- Preloaded asset list with at least five assets.
- Create a new asset and reject duplicate asset codes.
- Open an asset details page or modal.
- Generate a working QR code from the asset public URL or asset code, display it, and provide an Open Asset Page action.
- Report an issue against an asset.
- Change issue status through the available workflow.
- Add a maintenance note and resolve the issue.
- Automatically add activity entries to the asset history.
- Search and filter assets or issues.
- Save data in localStorage so it survives refresh.
- Reset Demo Data option.
- Responsive interface.

8. Evaluation and Marking Bonuses

Bonus Area	Maximum Bonus
AWS deployment	5
Docker	3
GitHub Actions CI/CD	4
Redis caching or justified Redis use	3
Email notifications or OTP	2
Rate limiting	1
Total Bonus	20

9. Submission Requirements

- GitHub repository link (both FE and BE for final hackathon students)
- Deployed application link
- Backend/API link where applicable
- Demo credentials
- README
- API documentation or Postman collection for Advanced Track
- Database schema or data-model diagram for database tracks
- Short demo video or live demo readiness upload on Linkedin (provide the link)

10. What Makes a Strong Submission?

Weak Submission	Strong Submission
QR generator with asset CRUD	Complete asset-to-issue-to-maintenance-to-history workflow
Frontend hides buttons for roles	Backend enforces authorization
AI chatbot box	Focused structured AI issue triage
AI response saved blindly	User reviews, edits, validates, and confirms output
Decorative dashboard charts	Useful operational summaries and filters
All code in one file	Clear modules, services, components, and responsibilities
One final Git commit	Incremental meaningful commits
Bonus tech added without purpose	Queue, Redis, email, and rate limiting applied to real product needs
Polished UI with broken workflow	Reliable end-to-end product with clean UX

16. Final Task Statement for Students

Build MaintainIQ: an AI-powered QR Maintenance & Asset History Platform that gives physical assets a digital identity, allows users to report issues, helps teams intelligently triage and resolve problems, and preserves a complete maintenance history.

Your assigned track defines the technologies and complexity expected from you. Complete the required core workflow before attempting optional features. You may use AI assistance, but you must declare its use and demonstrate full ownership of your implementation during live evaluation.

The goal is not to create the largest number of screens. The goal is to create a reliable, usable, sellable product that demonstrates product thinking, programming fundamentals, engineering decisions, debugging ability, AI integration, and ownership of code.